

Módulo 5: Herramientas low-code y no-code con IA (42 h)

Reglas e instrucciones comunes

1. **Siempre trabajar con un proceso realista, no con una “demo suelta”:** entrada → transformación → decisión → salida medible. (El módulo insiste en que low-code/no-code con IA es **orquestración de procesos**, no “usar ChatGPT suelto”).
2. **Contrato de datos obligatorio (JSON):** cada práctica debe definir **campos mínimos, campos opcionales**, tipos (texto/número/booleano/fecha), listas y metadatos (source, timestamp, trace_id).
3. **Rutas separadas:**
 - Ruta principal (OK)
 - Ruta de revisión (faltan datos / caso sensible / salida IA dudosa)
 - Ruta de error técnico (401/403/429/5xx, timeouts)
4. **Validación y fallback:** si falla el contrato de datos o la salida IA no cumple formato → reintento limitado o derivación a revisión humana.
5. **Trazabilidad mínima:** todo entregable debe incluir un **trace_id**, qué se registra en logs y 4–6 KPIs (tiempo de ciclo, tasa de éxito, revisión, error técnico, retrabajo, volumen).
6. **Datos ficticios y privacidad:** no se usan datos reales de clientes; se simulan pedidos, nombres y emails.
7. **Entrega estándar:** 1–2 páginas + tablas/diagramas + evidencias (capturas, export del escenario/workflow o “pseudo-capturas” si no se dispone de la herramienta).

Práctica 1 — Selección del “caso troncal” y mapa AS-IS → TO-BE

Tipo: Grupal (4–5)

Encargo (necesidad de empresa):

Zalando (e-commerce) quiere reducir el tiempo que su equipo de soporte dedica a **clasificar y registrar incidencias** de pedidos (devolución, entrega fallida, cambio de talla, facturación). El volumen es alto y la información llega desordenada por email/formulario: a veces falta número de pedido, otras el cliente mezcla temas (“no me llegó + quiero devolución + cambio de dirección”). La empresa pide un primer paso: elegir un caso “troncal” automatizable y describirlo con precisión para poder implementarlo después en Make o n8n.

Enlaces para analizar la empresa (proceso y soporte):

Ayuda y contacto Zalando:

<https://www.zalando.es/ayuda/>

Devoluciones Zalando (para entender reglas típicas):

<https://www.zalando.es/ayuda/Devoluciones-y-reembolsos/>

Lo que se pide (propuesta práctica):

1. Elegir 1 proceso troncal (p. ej., “devolución y reembolso por incidencia”).
2. Construir **AS-IS** (cómo se hace hoy) y **TO-BE** (con automatización + control).
3. Definir alcance (incluye/excluye) y puntos de control humano. (Selección y AS-IS/TO-BE es núcleo del 5.1.4).

Herramientas recomendadas: Miro / draw.io / PowerPoint + Docs.

Requerimientos mínimos:

- Matriz rápida valor/riesgo/datos/complexidad del proceso elegido
- AS-IS con 8–12 pasos y TO-BE con rutas OK/revisión/error
- Lista de excepciones (mín. 6)

Resultado esperado (entregable):

Mapa AS-IS/TO-BE + ficha del caso troncal (1–2 páginas) lista para construir en Make/n8n.

Práctica 2 – Contrato de datos (JSON) + validación (entrada/salida)

Tipo: Individual

Encargo (necesidad de empresa):

Una **gestoría** recibe solicitudes por email para “alta de autónomo”, “cambio de domicilio fiscal”, “certificados”, etc. El problema: cada email trae datos distintos, y el equipo pierde tiempo pidiendo lo mismo una y otra vez. Quieren normalizar la entrada a un **JSON estándar** antes de automatizar nada. (Esto ataca el “80% de problemas vienen de datos” del módulo).

Enlaces para analizar (referencia de servicios típicos, para inspirar campos):

Ejemplo de gestión (servicios y trámites frecuentes):

<https://www.gestoriaonline.com/>

Lo que se pide:

1. Diseñar un **JSON contrato** para “solicitud de trámite” con: campos mínimos, opcionales, tipos, arrays (adjuntos), metadatos.
2. Definir **reglas de validación** (entrada y salida) y mensajes de “faltan datos”.
3. Diseñar la ruta “datos faltantes” (qué preguntas mínimas hace el sistema antes de crear ticket).

Herramientas recomendadas: Docs + (opcional) validador JSON online / Sheets.

Requerimientos mínimos:

- Campos mínimos: tipo_tramite, email_contacto, nombre_cliente, resumen, prioridad, attachments[]
- Metadatos: source, timestamp ISO, trace_id, schema_version
- 10 reglas de validación (ej.: prioridad ∈ {alta, media, baja})

Resultado esperado:

Documento con: JSON “plantilla”, 3 ejemplos de entrada (buen caso / faltan datos / caso sensible) y reglas de validación.

Práctica 3 — Decidir el trigger: evento vs programación vs polling (con KPI y riesgo)

Tipo: Individual

Encargo (necesidad de empresa):

Glovo recibe incidencias operativas en tiempo real (pedidos retrasados, entrega fallida, reclamaciones), pero también necesita consolidar informes diarios para operaciones.

Quieren automatizar dos cosas:

- A) un flujo “en caliente” para incidencias, y
- B) un resumen diario de incidencias por zona.

La decisión clave: **cómo se dispara** cada flujo (evento / cron / polling) y qué riesgos genera.

Enlaces para analizar:

Centro de ayuda Glovo:

<https://help.glovoapp.com/>

Lo que se pide:

Proponer **2 flujos** (A y B) y justificar: trigger elegido, latencia esperada, riesgo de duplicados, coste por ejecuciones y KPI principal. (El módulo enfatiza decisión evento vs batch y riesgos).

Herramientas recomendadas: Plantilla (tabla) + Docs.

Requerimientos mínimos:

- Para cada flujo: trigger, entrada, salida, KPI, riesgo y mitigación
- 1 decisión de “idempotencia” (cómo evitar duplicados)

Resultado esperado:

Tabla comparativa A/B (1 página) lista para pasar a implementación.

Práctica 4 — Make vs n8n: matriz de decisión + defensa (taller)

Tipo: Grupal

Encargo (necesidad de empresa):

Una empresa de **formación online** (tipo Domestika) quiere automatizar: lead web → clasificación → CRM → email de bienvenida → creación de tarea comercial. El equipo duda:

- Make parece rápido para montar,
 - n8n interesa por flexibilidad y gobierno (y posibilidad de alojarlo, si hiciera falta).
- Dirección pide una decisión **defendible**: criterios con pesos, trade-offs y plan de mitigación.

Enlaces para analizar (referencia de funcionamiento/lead funnel):

Domestika (catálogo y modelo de captación):

<https://www.domestika.org/es>

Lo que se pide:

1. Matriz de decisión Make vs n8n con 8 criterios mínimos (datos/listas, HTTP, observabilidad, gobierno, coste/volumen, seguridad/hosting, time-to-value...).
2. Puntuación 1-5 por criterio + peso.
3. Argumentario oral de 2 minutos + respuestas a 3 preguntas difíciles (coste real, qué pasa si falla, quién mantiene).

Herramientas recomendadas: Sheets + Slides.

Requerimientos mínimos:

- Tabla "criterio → implicación → decisión → riesgo → mitigación → KPI" (formato del módulo)
- 3 trade-offs explícitos ("acepto X para ganar Y")

Resultado esperado:

Matriz + 2 diapositivas de defensa + hoja de riesgos/mitigaciones.

Práctica 5 — Make (nivel 1): primer escenario reproducible con salida visible

Tipo: Grupal

Encargo (necesidad de empresa):

SEUR quiere automatizar un primer paso muy acotado: cuando entra un formulario de "incidencia de envío", crear un registro interno (sheet) y avisar a un responsable. Solo piden un MVP reproducible, con trazabilidad y control de datos mínimos antes de crecer a ticketing real.

Enlaces:

SEUR (servicios y seguimiento, para inspirar campos):

<https://www.seur.com/>

Lo que se pide:

Construir (o diseñar si no hay acceso a herramienta) un escenario Make con:

- Trigger (Webhook o Watch)
- Validación de campos mínimos
- Salida visible (fila en sheet o email)
- trace_id incluido
- Evidencia de 3 ejecuciones reproducibles (misma entrada → salida esperada).

Herramientas recomendadas: Make + Google Sheets/Email.

Requerimientos mínimos:

- 1 ruta OK y 1 ruta "faltan datos"
- 3 entradas de prueba (OK / faltan datos / caso ambiguo)

Resultado esperado:

Capturas + export del escenario + tabla con resultados de prueba.

Práctica 6 — Make (nivel 2): Router/Filters + ruta de revisión (humano en el bucle)

Tipo: Grupal

Encargo (necesidad de empresa):

Una empresa de **software B2B** (tipo Typeform) recibe leads muy variados. No todos merecen atención comercial: algunos son estudiantes, otros piden “prueba gratis”, otros son empresas grandes con urgencia. Quieren que la automatización enrute:

- **Alta prioridad** → comercial senior + aviso inmediato
- **Media** → cola estándar
- **Baja** → nurturing (email informativo)
- **Sensible/ambiguo** → **revisión humana** antes de actuar.

Enlaces para analizar (modelo de captación y contacto):

Typeform (contacto/solicitud demo):

<https://www.typeform.com/>

Lo que se pide:

En Make: construir lógica de Router/Filters con reglas claras de prioridad, ruta de revisión y ruta de descarte (spam/duplicado), colocando filtros antes de pasos caros.

Herramientas recomendadas: Make + Sheets + Email/Slack/Teams.

Requerimientos mínimos:

- 4 rutas (alta/media/baja/revisión) + justificación de reglas
- 15 casos de prueba (5 normales, 5 incompletos, 5 límite)

Resultado esperado:

Escenario + documento de reglas + set de pruebas con resultado esperado.

Práctica 7 — Make (nivel 3): listas, Iterator/Aggregator y control de coste

Tipo: Grupal

Encargo (necesidad de empresa):

IKEA gestiona incidencias donde un mismo caso puede traer **varios artículos** y **varios adjuntos** (fotos, tickets). Quieren automatizar el registro sin “explotar” operaciones: si un caso trae 30 fotos, no quieren disparar 30 acciones sin control. El objetivo es aprender a tratar **listas** y a decidir: iterar, agregar o pasar a revisión.

Enlaces:

IKEA – devoluciones y reclamaciones:

<https://www.ikea.com/es/es/customer-service/returns-claims/>

Lo que se pide:

Diseñar/implementar un flujo que:

- Detecta arrays (items[], attachments[])
- Si attachments <= 5: procesa (Iterator)

Programa Superior en Inteligencia Artificial Aplicada a la Empresa

Guía de prácticas

- Si attachments > 5: agrega resumen (Aggregator) + deriva a revisión
- Registra en log cuántos elementos había (KPI de volumen).

Herramientas recomendadas: Make + Sheets/Drive.

Requerimientos mínimos:

- Regla “si lista supera X → revisión”
- Evidencia de 3 escenarios: 0 adjuntos / 3 adjuntos / 12 adjuntos

Resultado esperado:

Diagrama + capturas de ejecución + tabla de “coste estimado” (nº de operaciones evitadas).

Práctica 8 — Integración universal por HTTP: autenticación y manejo de errores

Tipo: Individual

Encargo (necesidad de empresa):

Una pyme usa un sistema de tickets poco conocido (sin conector). Necesita integrar: “crear ticket” y “consultar estado”. La API devuelve errores típicos (401/403/429/5xx).

Dirección exige: reintentos limitados, backoff y alertas cuando la integración falla. (Esto está en el bloque de integración y operación mínima).

Enlaces para analizar (API pública de ejemplo para practicar conceptos):

JSONPlaceholder (API de pruebas para practicar GET/POST):

<https://jsonplaceholder.typicode.com/>

Lo que se pide:

1. Diseñar una llamada HTTP (GET y POST) con cabeceras y parsing de respuesta JSON.
2. Definir qué ocurre con 401/403/429/5xx (ruta error técnico separada).
3. Plan de reintentos + backoff + alerta.

Herramientas recomendadas: Make HTTP module o n8n HTTP Request (según disponibilidad) + documento.

Requerimientos mínimos:

- Tabla de tratamiento por código de error (401/403/429/5xx)
- trace_id y log mínimo (qué se registra)

Resultado esperado:

Especificación técnica breve (1–2 páginas) lista para implementar.

Práctica 9 — n8n: replicar el caso en “modo equivalente” (items, IF/Switch, Split in Batches)

Tipo: Grupal

Encargo (necesidad de empresa):

Programa Superior en Inteligencia Artificial Aplicada a la Empresa

Guía de prácticas

Una empresa quiere evitar “dependencia de herramienta”: pide que el mismo flujo (triage de incidencias) sea traducible entre Make y n8n, manteniendo lógica y contrato de datos. Se necesita comparar conceptos: bundle vs item, router vs IF/Switch, y control por lotes.

Enlaces (contexto de incidencias típico):

Correos (seguimiento / ayuda, inspiración de casos):

<https://www.correos.es/es/es/ayuda>

Lo que se pide:

1. Tomar el diseño del flujo de una práctica previa (p. ej., práctica 6) y traducirlo a n8n.
2. Implementar IF/Switch y una estrategia de lotes (Split In Batches) para evitar picos.
3. Documentar equivalencias funcionales (terminología) y riesgos de migración.

Herramientas recomendadas: n8n (cloud o local) + Docs.

Requerimientos mínimos:

- 1 ruta OK + 1 revisión + 1 error técnico
- Tabla de equivalencias Make ↔ n8n aplicada al flujo

Resultado esperado:

Workflow n8n + documento “equivalencias y decisiones” (1–2 páginas).

Práctica 10 – IA “operable” en el flujo: contrato de salida + validación + fallback + demo final

Tipo: Grupal (capstone del módulo)

Encargo (necesidad de empresa):

Aena recibe mensajes de pasajeros (email/formulario/redes) con incidencias y preguntas. Quieren un flujo que:

- Clasifique motivo (equipaje, accesibilidad, información, queja)
- Extraiga datos (aeropuerto, fecha, vuelo si aparece, canal, urgencia)
- Genere un **borrador** de respuesta (NO envío automático)
- Si falta dato o hay señal sensible (denuncia, datos personales, menor), deriva a revisión humana.

La prioridad es que la IA sea un **paso dentro del proceso**, con contrato de salida estricto y validación.

Enlaces para analizar:

Contacto Aena:

<https://www.aena.es/es/contacto.html>

Lo que se pide:

1. Definir **entrada JSON** y **salida JSON** del paso de IA (campos obligatorios, valores permitidos, “no_indicado” cuando falte).
2. Redactar un **prompt operativo** para clasificación/extracción/sugerencia de borrador con formato estricto.

Programa Superior en Inteligencia Artificial Aplicada a la Empresa

Guía de prácticas

3. Validación post-IA (si no cumple: reintento 1 vez; si vuelve a fallar: revisión).
4. Checklist de producción: credenciales, logs, alertas, control de cambios/export.
5. Demo obligatoria: 1 ejecución OK + 1 fallo tratado (como pide el cierre del módulo).

Herramientas recomendadas: Make o n8n + un LLM vía conector o HTTP + Sheets/Drive para logs.

Requerimientos mínimos:

- 20 casos de prueba: 8 OK, 6 incompletos, 6 límite/sensibles
- 4-6 KPIs (tiempo ciclo, % revisión, % error técnico, retrabajo, volumen)
- Evidencias: capturas/exports + documento final

Resultado esperado:

“Paquete de entrega” del mini-proyecto:

- diagrama del flujo + contrato JSON + prompt + validación/fallback
- checklist de producción + evidencia de demo (OK y fallo tratado)